ALGORITHME DE BELLMAN

F. M.

2022/2023

SOMMAIRE

1. REPRESENTATION D'UNE ARBORESCENCE
2. ORDRE TOPOLOGIQUE
3. PROPRIÉTÉ
4. L'ALGORITHME
5. VARIANTES

Table des pères et table des potentiels

Représentation d'une r-arborescence T incluse dans G par table des pères et table des potentiels :

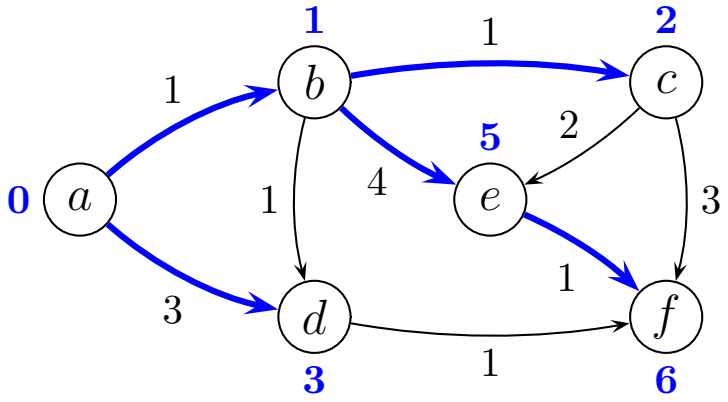
$$\forall s \in S,\ \mathcal{A}[s] = \begin{cases} \text{none} & \text{si } s = r \\ x & \text{si l'arc } (x, s) \in T \\ s & \text{si } s \notin T \end{cases}$$

$$\forall s \in S,\ \Pi[s] = \begin{cases} \text{poids du chemin de } r \text{ à } s \text{ dans } T & \text{si } s \in T \\ \infty & \text{si } s \notin T \end{cases}$$

Le sommet $\mathcal{A}[s]$ est appelé père du sommet s, le nombre $\Pi[s]$ est appelé potentiel du sommet s.

Illustration

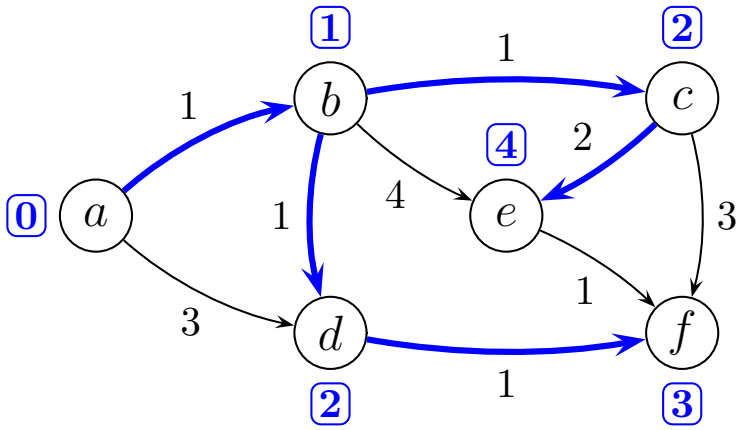
Une a-arborescence :



s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	a	b	a	b	e
$\Pi[s]$	0	1	2	3	5	6

$\Pi[s] \geq \textsc{Dist}(a, s)$

Une a-arborescence de cpm :



s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	a	b	b	c	d
$\Pi[s]$	0	1	2	2	4	3

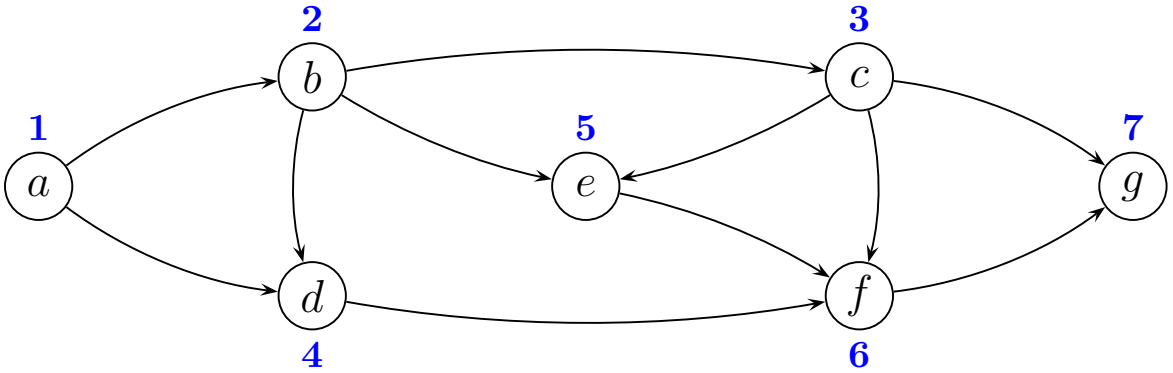
$\Pi[s] = \textsc{Dist}(a, s)$

Définition

Soit $G = (S, A)$ un graphe orienté d'ordre n. On appelle **ordre topologique** pour G toute bijection $\varphi : S \mapsto \{1, \ldots, n\}$ vérifiant :

$$\varphi(x) < \varphi(y), \ \forall (x, y) \in A$$

Le nombre $\boldsymbol{\varphi(x)}$ est appelé **numéro topologique** du sommet x.



Propriété

Un graphe orienté G est un graphe sans circuit ou DAG (Directed Acyclic Graph) si et seulement si il existe un ordre topologique pour G.

Algorithme de tri topologique

Donnée : G, un DAG
Résultat : φ, un ordre topologique pour G

$k \leftarrow 0$
tant que G *contient une source* s **faire**
  $\varphi(s) \leftarrow k \leftarrow k+1$ ..... ✎ numérotation topologique de s
  examiner(s) ..... ✎ un traitement du sommet s
  $G \leftarrow G - s$ ..... ✎ suppression de s dans G
fin tant que

Cet algorithme permet d'examiner les sommets dans un ordre topologique pour G. Assimilable à un *itérateur* sur l'ensemble des sommets.

Terminologie

On appelle **candidat** les sources du graphe obtenu à chaque itération.

3. PROPRIÉTÉ

Propriété 3.1

Soient $G = (S, A, P)$ un DAG pondéré et r un sommet de G. Alors, pour tout sommet s de S, $s \neq r$, on a :

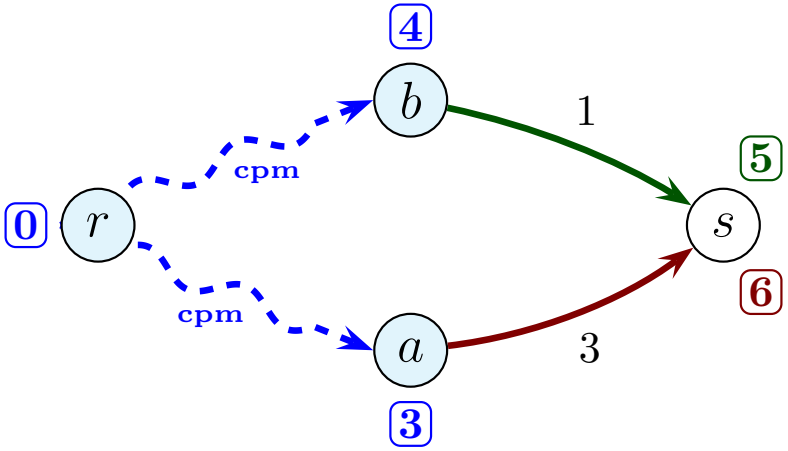
$$\textsc{Dist}(r, s) = \min\left\{\textsc{Dist}(r, x) + P(x, s) : x \in V^-(s)\right\}$$

Si il existe un cpm allant de r à s alors celui-ci passe nécessairement par un prédécesseur de s. Si les distances $\textsc{Dist}(r, x)$, pour tout $x \in V^-(s)$, sont connues alors la formule de récurrence ci-dessus permet de calculer la distance de r à s.

S'il n'existe aucun chemin de r à s, $r \neq s$, alors il n'existe pas non plus de chemin allant de r vers l'un ou l'autre des prédécesseurs de s. La formule ci-dessus donnera alors $\textsc{Dist}(r, s) = \infty$. Cela sera aussi le cas si $s \neq r$ est source.

Illustration de la propriété

Calcul de $\textsc{Dist}(r, s)$ à partir de $\textsc{Dist}(r, a)$ et $\textsc{Dist}(r, b)$:



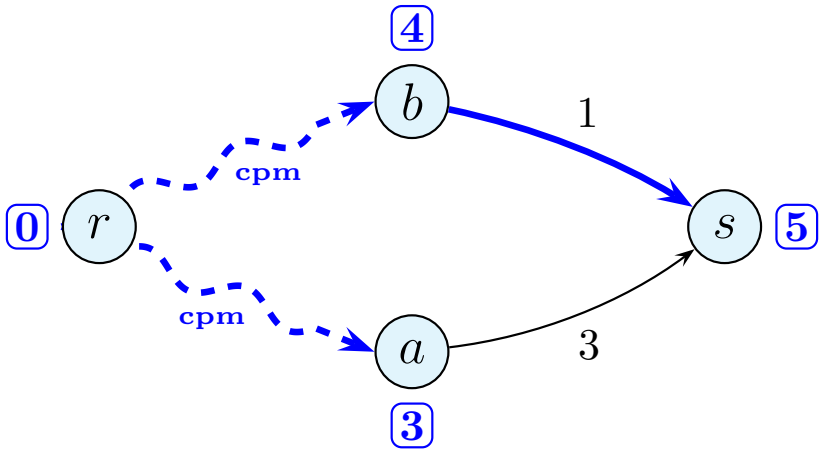
$V^-(s) = \{a, b\}$

$\textsc{Dist}(r, a) = 3$

$\textsc{Dist}(r, b) = 4$

$\textbf{Dist}(r, a) + P(a, s) = 3 + 3 = 6 \quad \textbf{Dist}(r, b) + P(b, s) = 4 + 1 = 5$

D'où $\textbf{Dist}(r, s) = 5$ et :



Algorithme de BELLMAN

Cet algorithme détermine une r-arborescence de cpm dans un DAG pondéré $G = (S, A, P)$. Cette arborescence est représentée par des tables Π et $\mathcal{A}$.

L'algorithme consiste à **examiner les sommets dans un ordre topologique** pour G. Examiner un sommet s consiste à :

- Si $s = r$, poser $\Pi[r] \leftarrow 0$ et $\mathcal{A}[r] \leftarrow$ none.
- Si $s \neq r$, relâcher tous les arcs de but s.

Tout sommet s est examiné après ses prédécesseurs. Relâcher tous les arcs de but s permet alors de calculer $\Pi[s] = \textsc{Dist}(r, s)$ à partir des distances $\Pi[x] = \textsc{Dist}(r, x)$, $x \in V^-(s)$, qui sont connues (formule de récurrence 3.1).

4. L'ALGORITHME

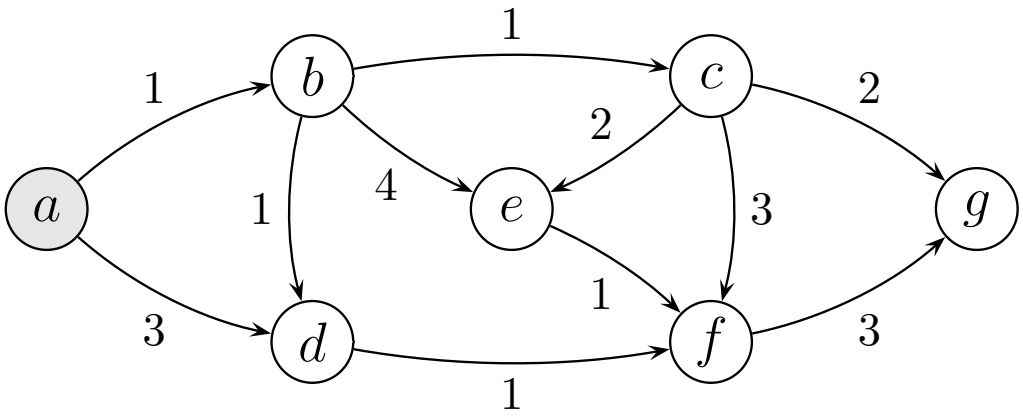
```
proc examiner(s)
    si s = r alors ..... ✎ cas où s est la racine
        Π[s] ← 0 ; A[s] ← none
    sinon ..... ✎ cas où s n'est pas la racine
        Π[s] ← ∞ ; A[s] ← s
        ✎ relâcher tous les arcs de but s
        pour tout x ∈ V⁻(s) faire
            si Π[x] + P(x, s) < Π[s] alors
                Π[s] ← Π[x] + P(x, s)
                A[s] ← x
            fin si
        fin pour
        ✎ ici, Π[s] = Dist(r, s),
    fin si
fin proc
```

Trace de l'algorithme : racine = unique source du DAG

itér [0]

racine : a

initialisation de l'ensemble des candidats



les premiers candidats sont les sources du graphe, le sommet a est le seul candidat

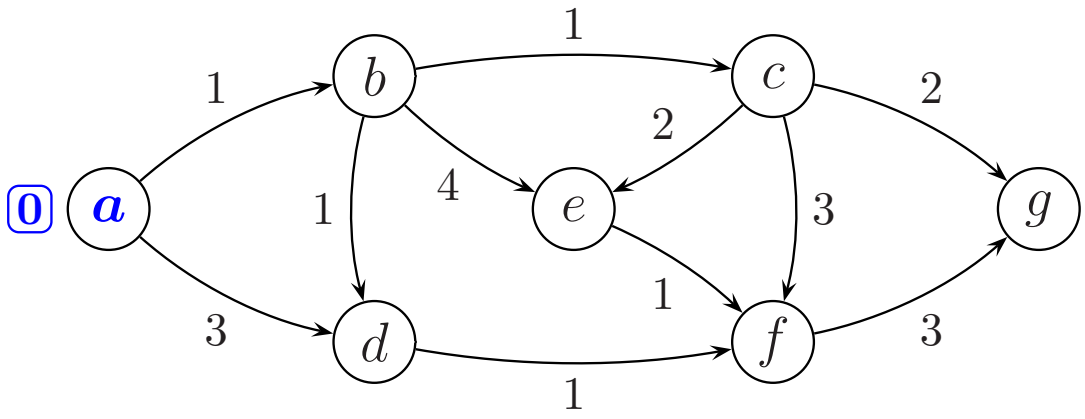
candidats	a
s	
$\Pi[s]$	
$\mathcal{A}[s]$	

Trace de l'algorithme : racine = unique source du DAG

itér [1]

examiner(a)

le sommet a est la racine choisie



$\boldsymbol{\Pi[a] \leftarrow 0,\ \mathcal{A}[a] \leftarrow \text{none}}$

à chaque relâchement d'un candidat, on annote le diagramme et on complète la colonne du tableau des résultats

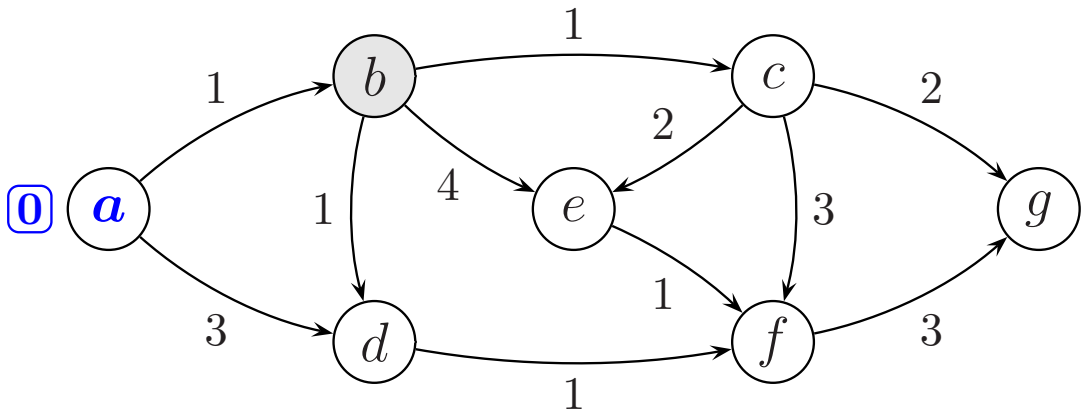
candidats	a
s	a
$\Pi[s]$	0
$\mathcal{A}[s]$	none

Trace de l'algorithme : racine = unique source du DAG

itér [1]

maj de l'ensemble des candidats

candidat = sommet non marqué ayant tous ses prédécesseurs marqués



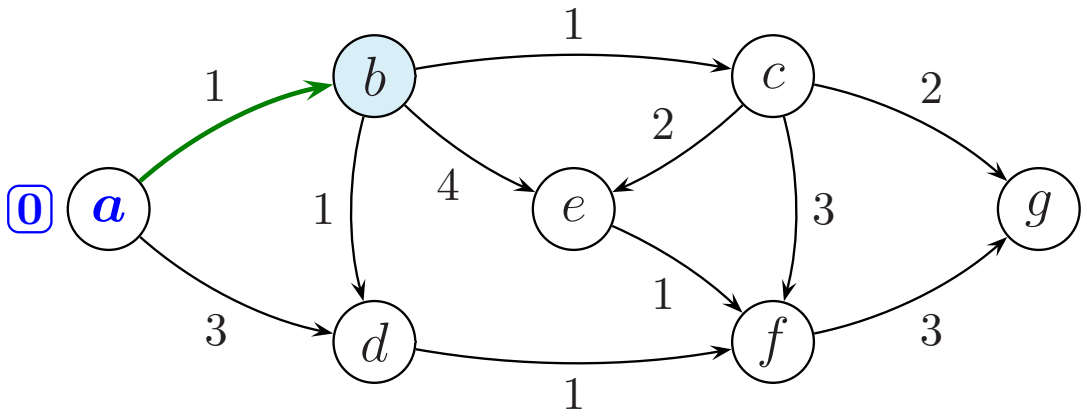
le sommet b devient candidat

candidats	a	b
s	a	
$\Pi[s]$	0	
$\mathcal{A}[s]$	none	

Trace de l'algorithme : racine = unique source du DAG

itér [2]

examiner(b)
$\Leftrightarrow$ relâcher l'arc (a, b)



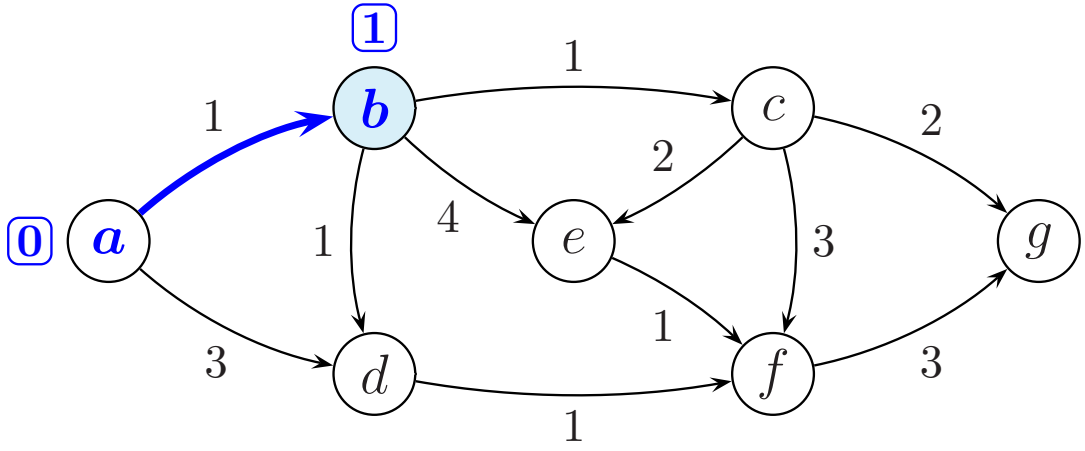
$$\underbrace{\Pi[a]}_{0} + \underbrace{P(a,b)}_{1} = 1$$

candidats	a	b
s	a	b
$\Pi[s]$	0	
$\mathcal{A}[s]$	none	

Trace de l'algorithme : racine = unique source du DAG

itér [2]

examiner(b)
$\Leftrightarrow$ relâcher l'arc (a, b)



$$\Pi[b] \leftarrow \underbrace{\Pi[a]}_{0} + \underbrace{P(a,b)}_{1} = 1$$

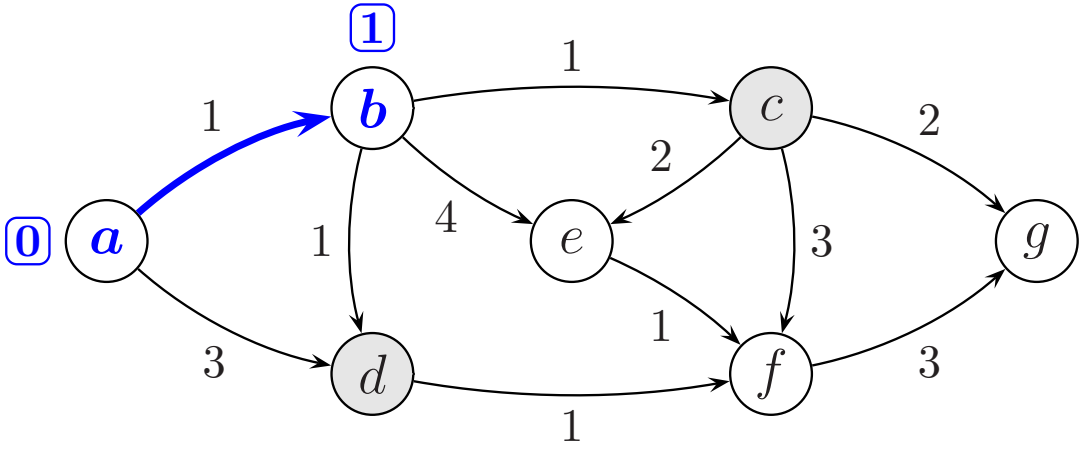
on colore en bleu, les arcs appartenant aux cpm

candidats	a	b
s	a	b
$\Pi[s]$	0	1
$\mathcal{A}[s]$	none	a

Trace de l'algorithme : racine = unique source du DAG

itér [2]

maj de l'ensemble des candidats



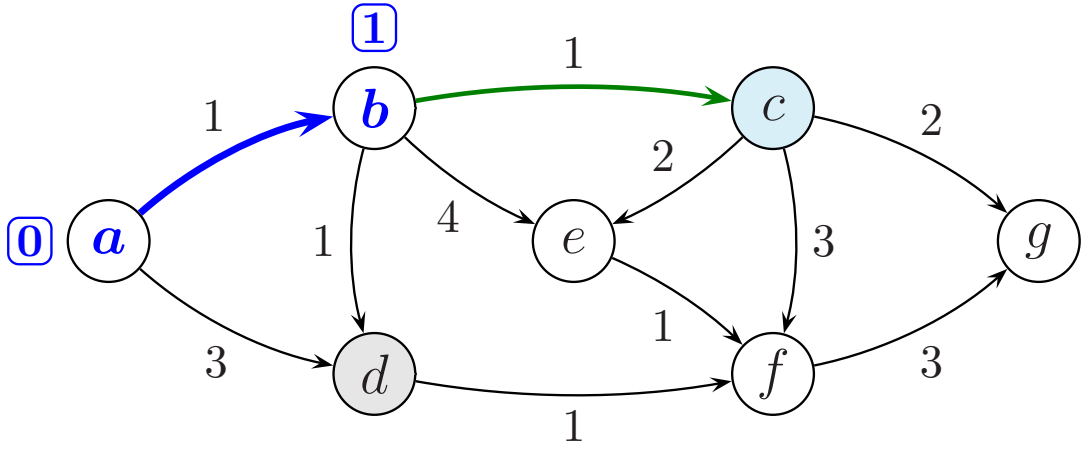
les sommets c et d deviennent candidats

candidats	a	b	c, d
s	a	b	
$\Pi[s]$	0	1	
$\mathcal{A}[s]$	none	a	

Trace de l'algorithme : racine = unique source du DAG

itér [3]

examiner(c)
$\Leftrightarrow$ relâcher l'arc (b, c)



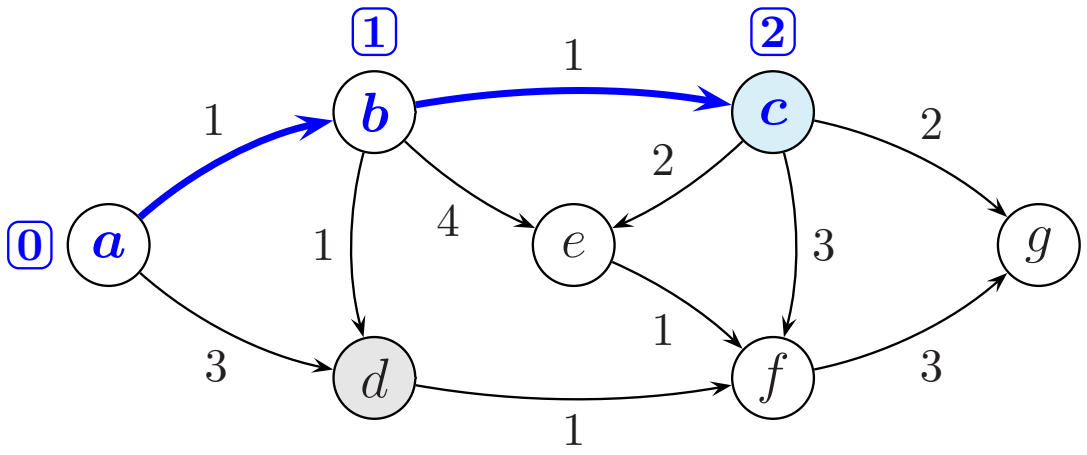
$$\underbrace{\Pi[b]}_{1} + \underbrace{P(b,c)}_{1} = 2$$

candidats	a	b	c, d
s	a	b	c
$\Pi[s]$	0	1	
$\mathcal{A}[s]$	none	a	

Trace de l'algorithme : racine = unique source du DAG

itér [3]

examiner(c)
$\Leftrightarrow$ relâcher l'arc (b, c)



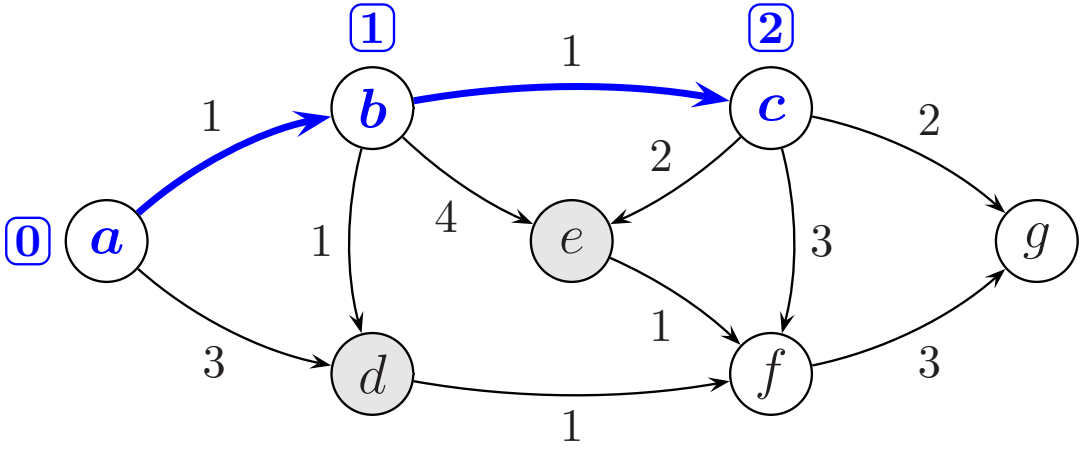
$$\Pi[c] \leftarrow \underbrace{\Pi[b]}_{1} + \underbrace{P(b,c)}_{1} = 2$$

candidats	a	b	c, d
s	a	b	c
$\Pi[s]$	0	1	2
$\mathcal{A}[s]$	none	a	b

Trace de l'algorithme : racine = unique source du DAG

itér [3]

maj de l'ensemble des candidats



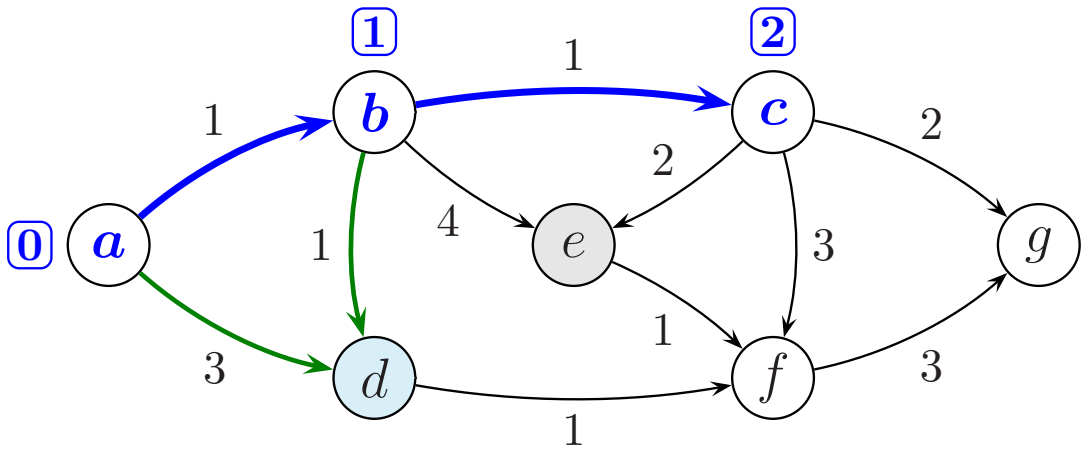
le sommet e devient candidat

candidats	a	b	c, d	d, e
s	a	b	c	
$\Pi[s]$	0	1	2	
$\mathcal{A}[s]$	none	a	b	

Trace de l'algorithme : racine = unique source du DAG

itér [4]

examiner(d)
$\Leftrightarrow$ relâcher les arcs (a, d) et (b, d)



$$\underbrace{\Pi[a]}_{0} + \underbrace{P(a,d)}_{3} = 3$$

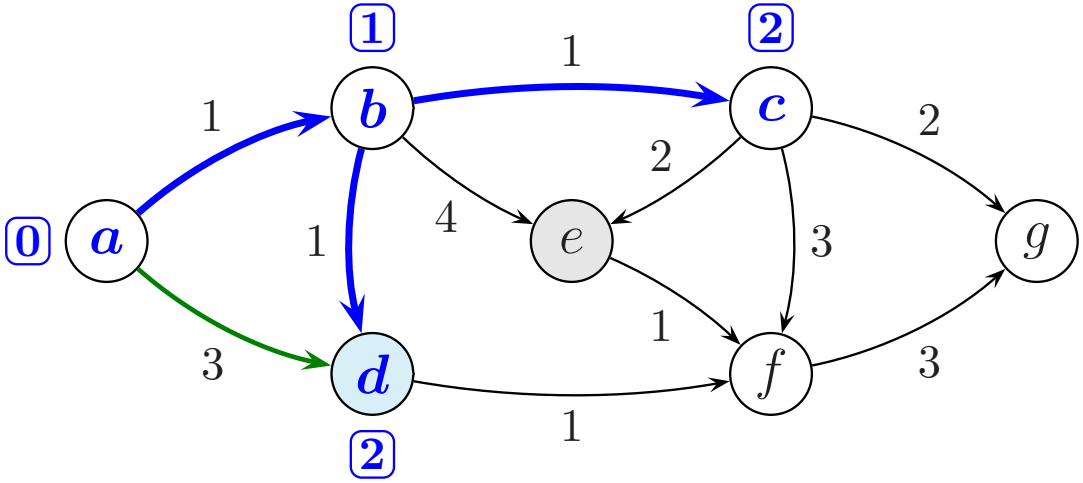
$$\underbrace{\Pi[b]}_{1} + \underbrace{P(b,d)}_{1} = 2$$

candidats	a	b	c, d	d, e
s	a	b	c	d
$\Pi[s]$	0	1	2	
$\mathcal{A}[s]$	none	a	b	

Trace de l'algorithme : racine = unique source du DAG

itér [4]

examiner(d)
$\Leftrightarrow$ relâcher les arcs (a, d) et (b, d)



$$\underbrace{\Pi[a]}_{0} + \underbrace{P(a,d)}_{3} = 3$$

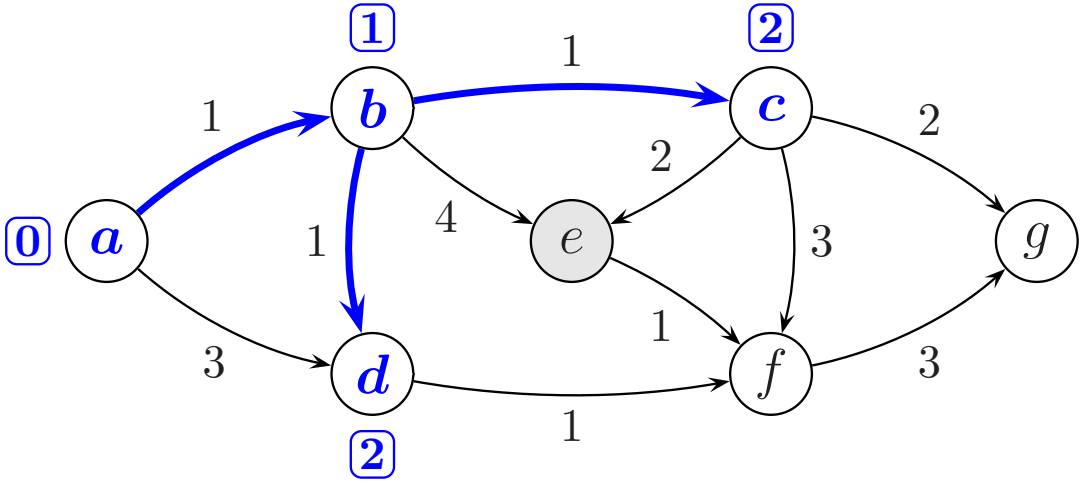
$$\Pi[d] \leftarrow \underbrace{\Pi[b]}_{1} + \underbrace{P(b,d)}_{1} = 2$$

candidats	a	b	c, d	d, e
s	a	b	c	d
$\Pi[s]$	0	1	2	2
$\mathcal{A}[s]$	none	a	b	b

Trace de l'algorithme : racine = unique source du DAG

itér [4]

maj de l'ensemble des candidats



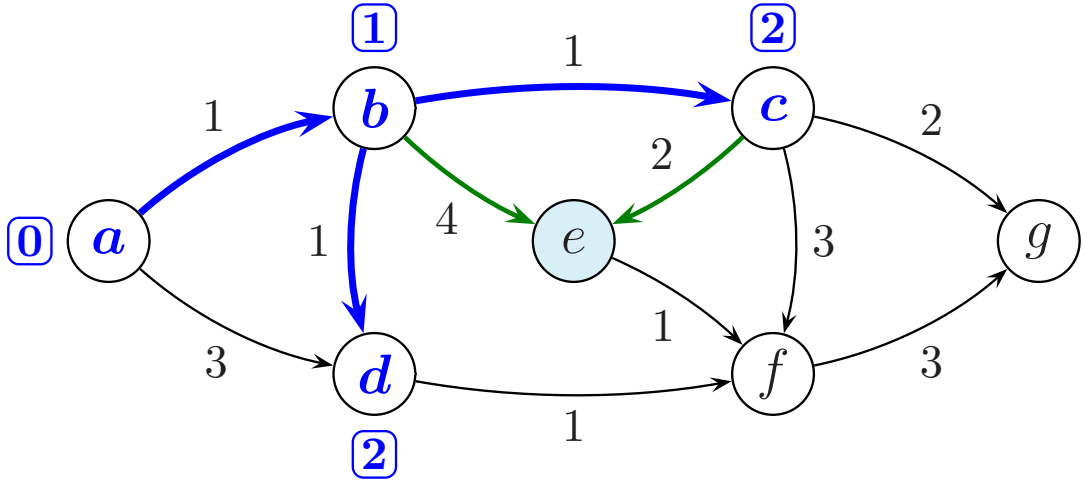
pas de nouveau candidat

candidats	a	b	c, d	d, e	e
s	a	b	c	d	
$\Pi[s]$	0	1	2	2	
$\mathcal{A}[s]$	none	a	b	b	

Trace de l'algorithme : racine = unique source du DAG

itér [5]

examiner(e)
$\Leftrightarrow$ relâcher les arcs (b, e) et (c, e)



$$\underbrace{\Pi[b]}_{1} + \underbrace{P(b,e)}_{4} = 5$$

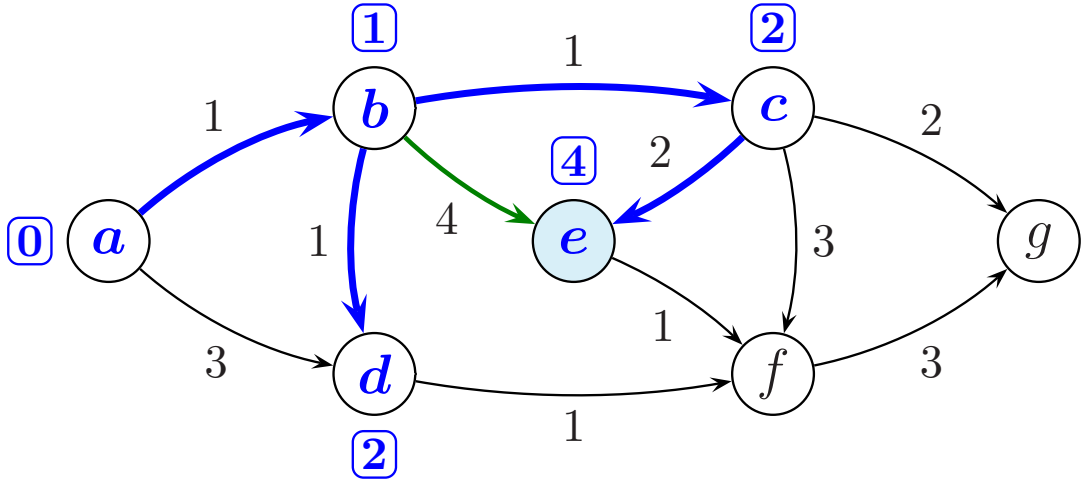
$$\underbrace{\Pi[c]}_{2} + \underbrace{P(c,e)}_{2} = 4$$

candidats	a	b	c, d	d, e	e
s	a	b	c	d	e
$\Pi[s]$	0	1	2	2	
$\mathcal{A}[s]$	none	a	b	b	

Trace de l'algorithme : racine = unique source du DAG

itér [5]

examiner(e)
$\Leftrightarrow$ relâcher les arcs (b, e) et (c, e)



$$\underbrace{\Pi[b]}_{1} + \underbrace{P(b,e)}_{4} = 5$$

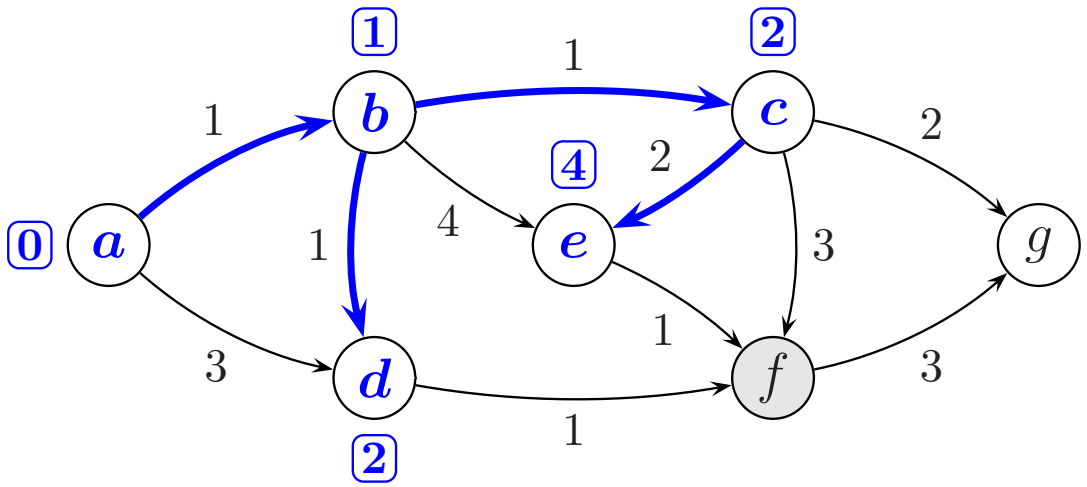
$$\Pi[e] \leftarrow \underbrace{\Pi[c]}_{2} + \underbrace{P(c,e)}_{2} = 4$$

candidats	a	b	c, d	d, e	e
s	a	b	c	d	e
$\Pi[s]$	0	1	2	2	4
$\mathcal{A}[s]$	none	a	b	b	c

Trace de l'algorithme : racine = unique source du DAG

itér [5]

maj de l'ensemble des candidats



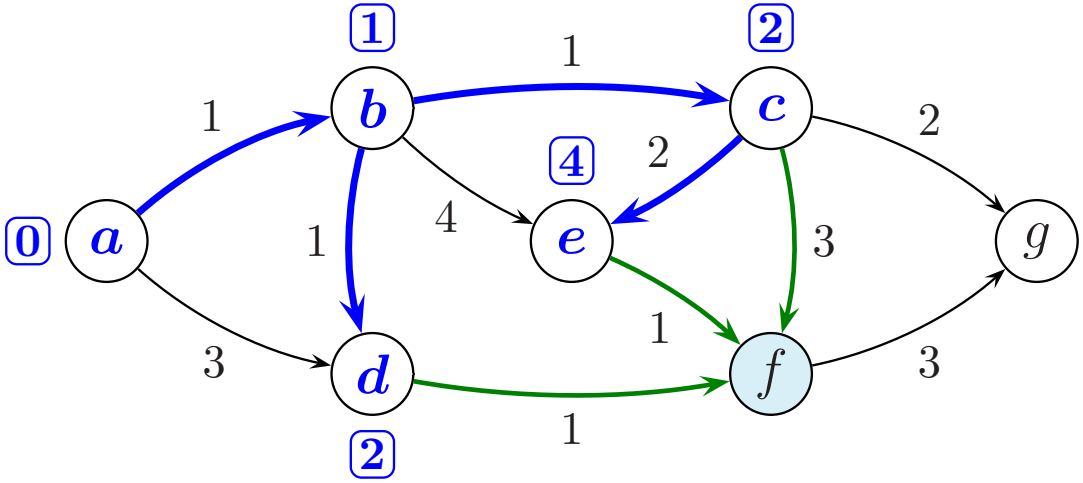
le sommet f devient candidat

candidats	a	b	c, d	d, e	e	f
s	a	b	c	d	e	
$\Pi[s]$	0	1	2	2	4	
$\mathcal{A}[s]$	none	a	b	b	c	

Trace de l'algorithme : racine = unique source du DAG

itér [6]

examiner(f)
$\Leftrightarrow$ relâcher les arcs (c, f), (d, f) et (e, f)



$$\underbrace{\Pi[c]}_{2} + \underbrace{P(c, f)}_{3} = 5$$

$$\underbrace{\Pi[d]}_{2} + \underbrace{P(d, f)}_{1} = 3$$

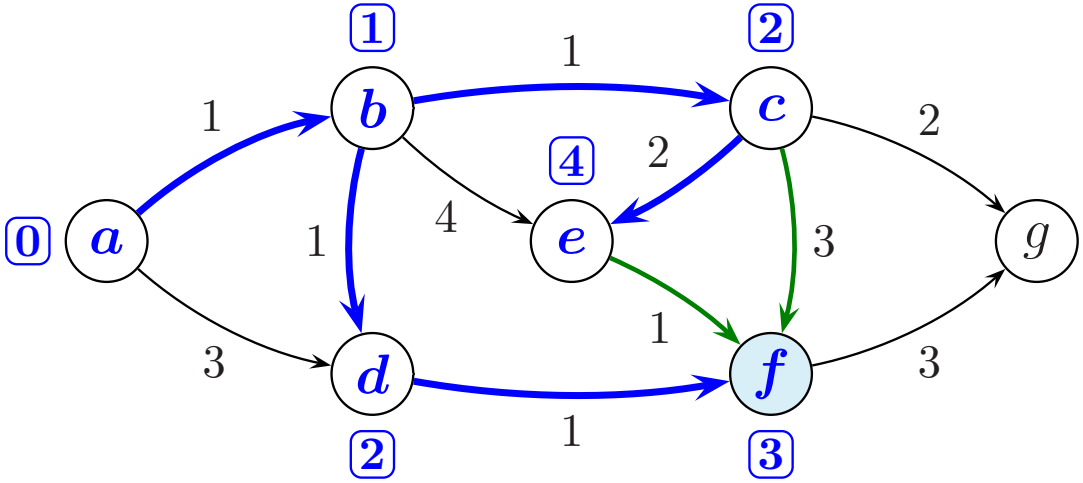
$$\underbrace{\Pi[e]}_{4} + \underbrace{P(e, f)}_{1} = 5$$

candidats	a	b	c, d	d, e	e	f
s	a	b	c	d	e	f
$\Pi[s]$	0	1	2	2	4	
$\mathcal{A}[s]$	none	a	b	b	c	

Trace de l'algorithme : racine = unique source du DAG

itér [6]

examiner(f)
$\Leftrightarrow$ relâcher les arcs (c, f), (d, f) et (e, f)



$$\underbrace{\Pi[c]}_{2} + \underbrace{P(c, f)}_{3} = 5$$

$$\Pi[f] \leftarrow \underbrace{\Pi[d]}_{2} + \underbrace{P(d, f)}_{1} = 3$$

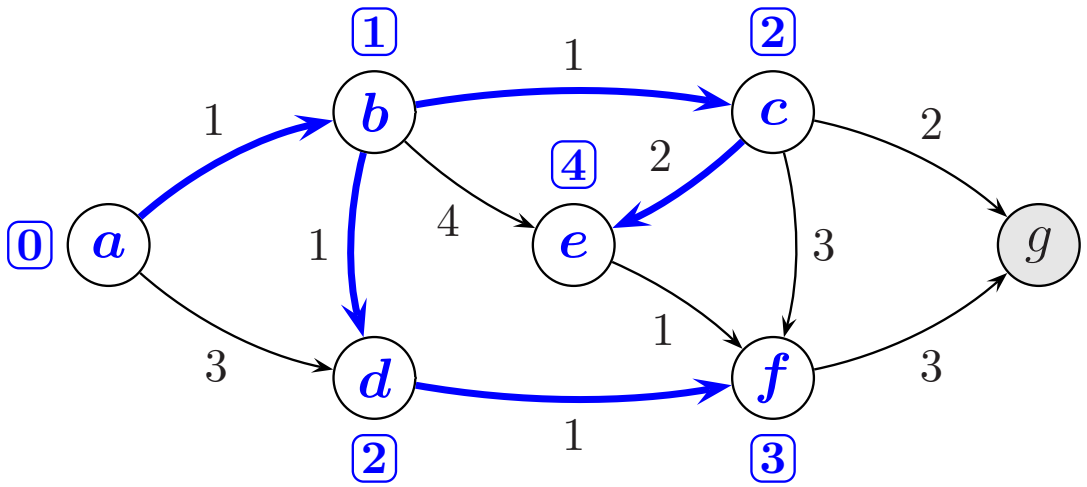
$$\underbrace{\Pi[e]}_{4} + \underbrace{P(e, f)}_{1} = 5$$

candidats	a	b	c, d	d, e	e	f
s	a	b	c	d	e	f
$\Pi[s]$	0	1	2	2	4	3
$\mathcal{A}[s]$	none	a	b	b	c	d

Trace de l'algorithme : racine = unique source du DAG

itér [6]

maj de l'ensemble des candidats



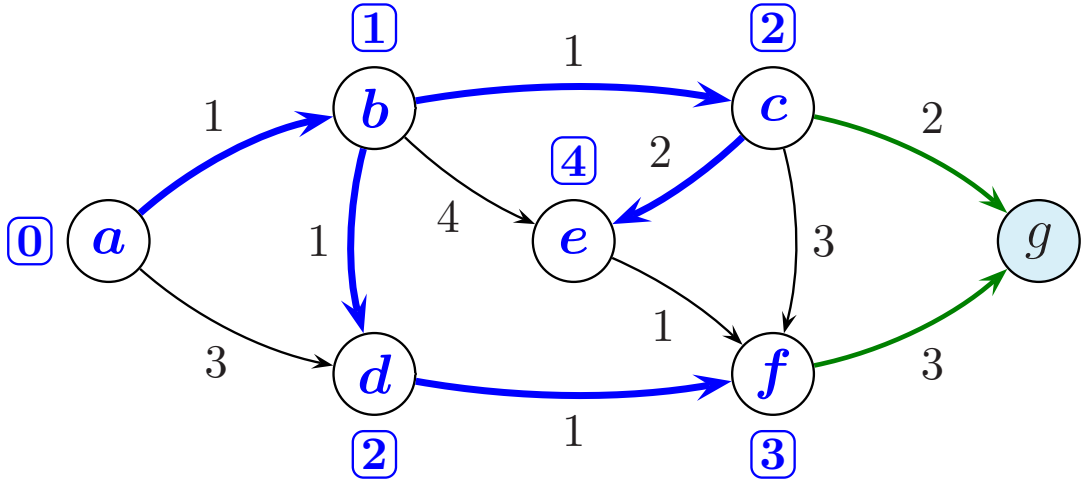
le sommet g devient candidat

candidats	a	b	c, d	d, e	e	f	g
s	a	b	c	d	e	f	
$\Pi[s]$	0	1	2	2	4	3	
$\mathcal{A}[s]$	none	a	b	b	c	d	

Trace de l'algorithme : racine = unique source du DAG

itér [7]

examiner(g)
$\Leftrightarrow$ relâcher les arcs (c, g) et (f, g)



$$\underbrace{\Pi[c]}_{2} + \underbrace{P(c,g)}_{2} = 4$$

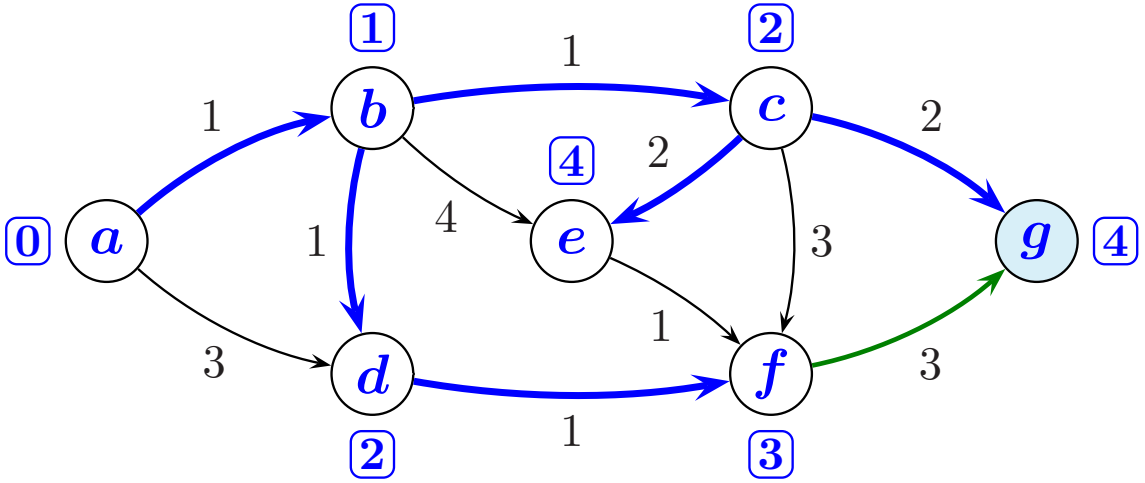
$$\underbrace{\Pi[f]}_{3} + \underbrace{P(f,g)}_{3} = 6$$

candidats	a	b	c, d	d, e	e	f	g
s	a	b	c	d	e	f	g
$\Pi[s]$	0	1	2	2	4	3	
$\mathcal{A}[s]$	none	a	b	b	c	d	

Trace de l'algorithme : racine = unique source du DAG

itér [7]

examiner(g)
$\Leftrightarrow$ relâcher les arcs (c, g) et (f, g)



$$\underbrace{\Pi[c]}_{2} + \underbrace{P(c,g)}_{2} = 4$$

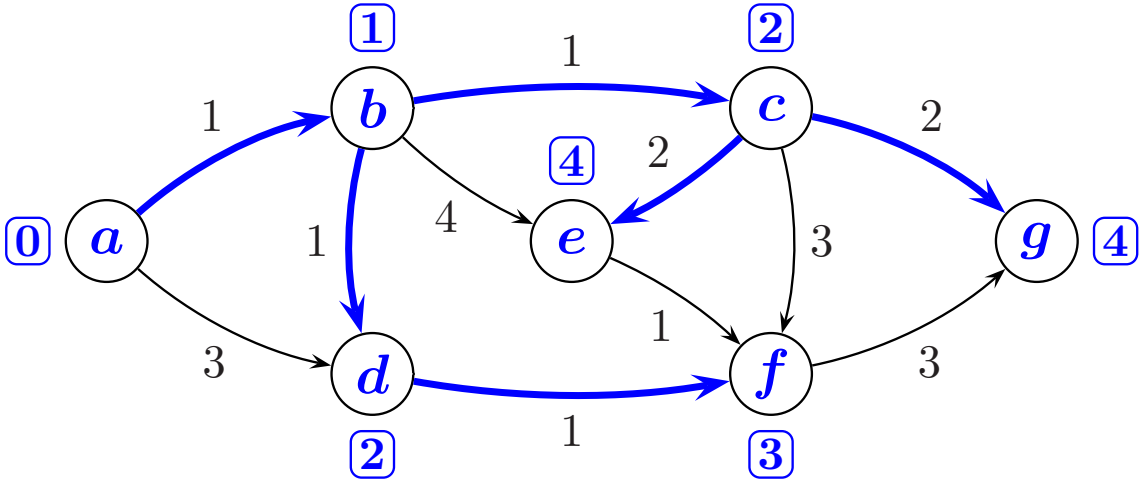
$$\Pi[g] \leftarrow \underbrace{\Pi[f]}_{3} + \underbrace{P(f,g)}_{3} = 6$$

candidats	a	b	c, d	d, e	e	f	g
s	a	b	c	d	e	f	g
$\Pi[s]$	0	1	2	2	4	3	4
$\mathcal{A}[s]$	none	a	b	b	c	d	c

Trace de l'algorithme : racine = unique source du DAG

itér [6]

maj de l'ensemble des candidats



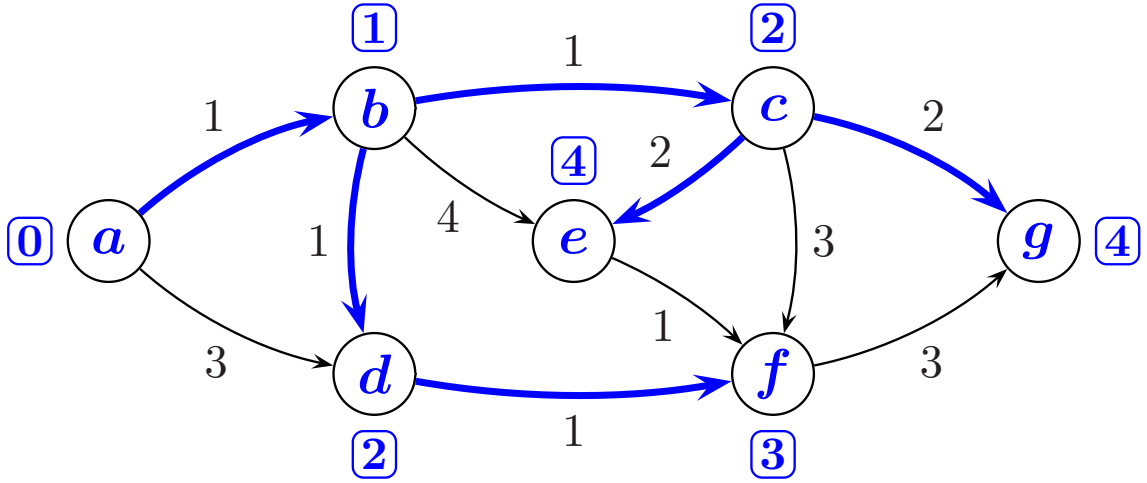
pas de nouveau candidat

candidats	a	b	c, d	d, e	e	f	g
s	a	b	c	d	e	f	g
$\Pi[s]$	0	1	2	2	4	3	4
$\mathcal{A}[s]$	none	a	b	b	c	d	c

Trace de l'algorithme : racine = unique source du DAG

résultat

l'ensemble des candidats est vide : fin de l'algorithme



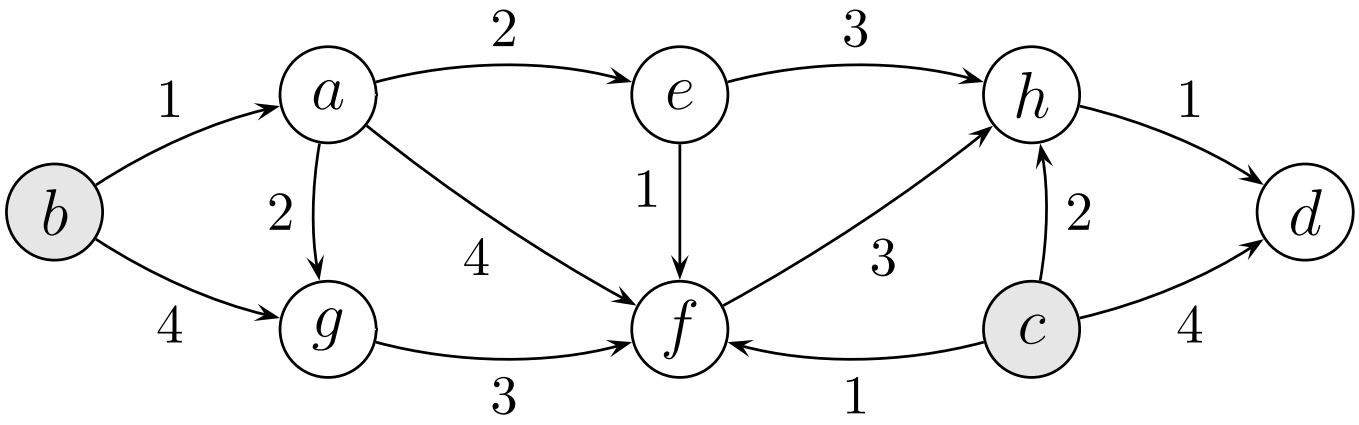
***a*-arborescence de cpm**

candidats	a	b	c, d	d, e	e	f	g
s	a	b	c	d	e	f	g
$\Pi[s]$	0	1	2	2	4	3	4
$\mathcal{A}[s]$	none	a	b	b	c	d	c

Trace de l'algorithme : racine ≠ source

itér [0]

racine : a ; a n'est pas une source

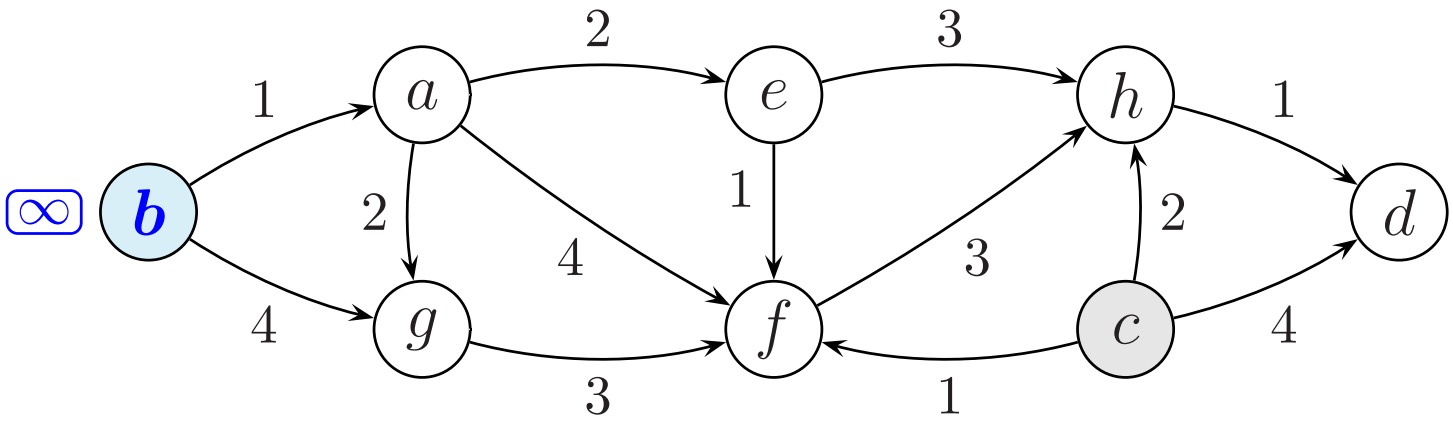


candidats	b, c
s	
$\Pi[s]$	
$\mathcal{A}[s]$	

Trace de l'algorithme : racine ≠ source

itér [1]

examiner(b) : aucun arc à relâcher car $V^-(b) = \varnothing$



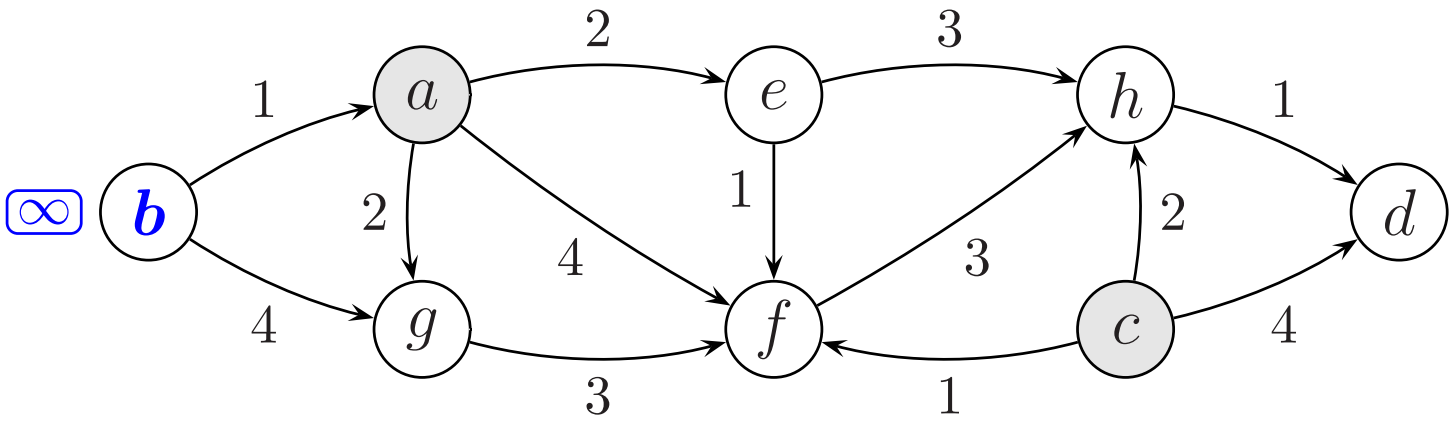
$\Pi[b] \leftarrow \infty \qquad \mathcal{A}[b] \leftarrow b$

candidats	b, c
s	b
$\Pi[s]$	∞
$\mathcal{A}[s]$	b

Trace de l'algorithme : racine ≠ source

itér [1]

mise à jour de l'ensemble des candidats



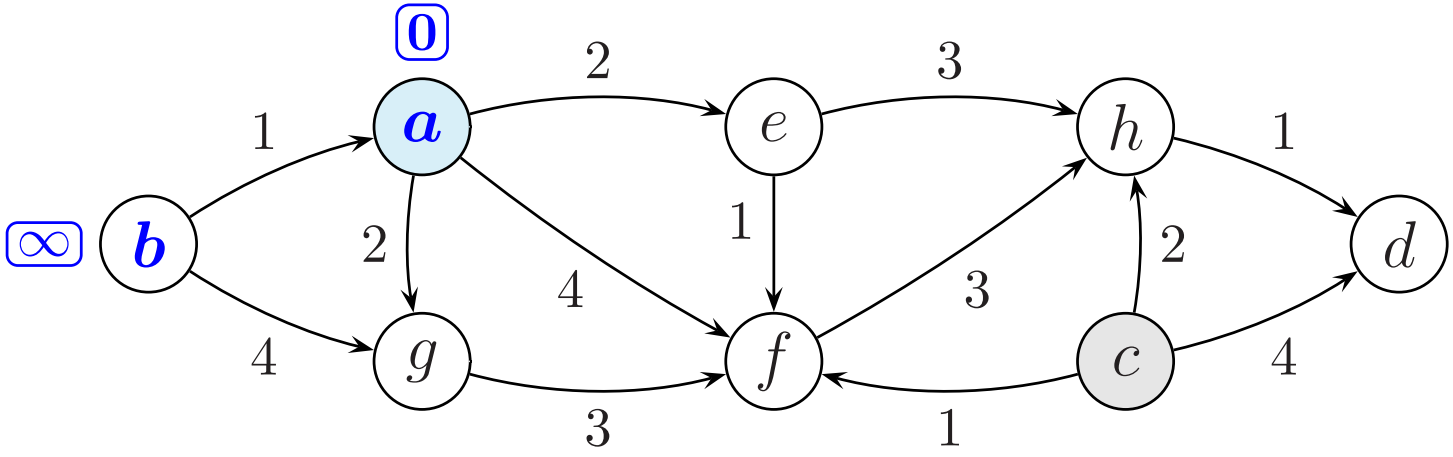
le sommet a devient candidat

candidats	b, c	a, c
s	b	
$\Pi[s]$	∞	
$\mathcal{A}[s]$	b	

Trace de l'algorithme : racine ≠ source

itér [2]

examiner(a) : a est la racine choisie



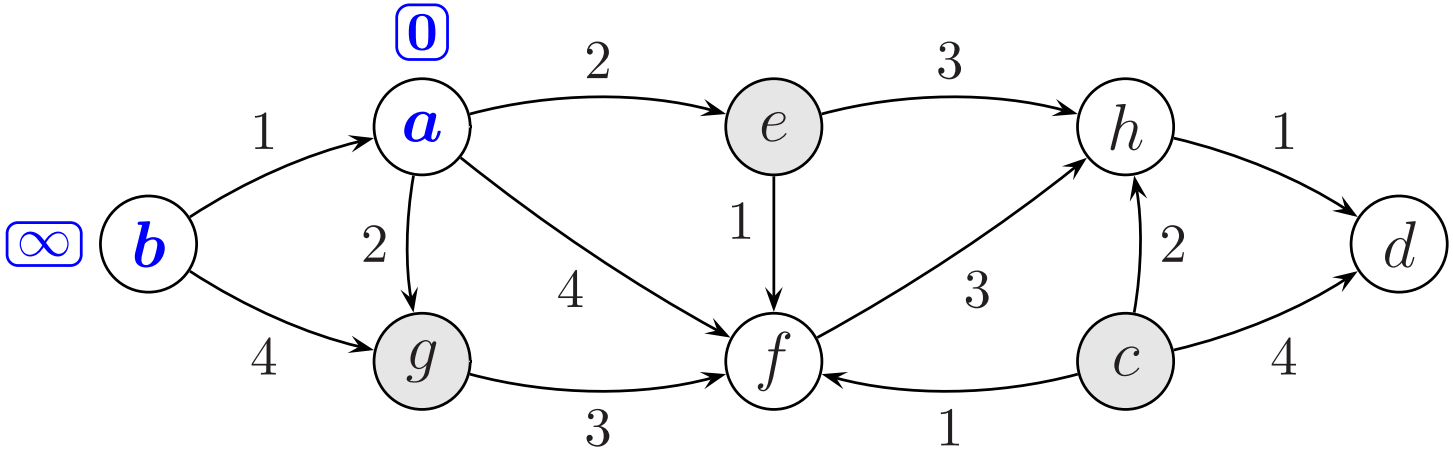
$\Pi[a] \leftarrow 0$ $\mathcal{A}[a] \leftarrow$ none

candidats	b, c	a, c
s	b	a
$\Pi[s]$	∞	0
$\mathcal{A}[s]$	b	none

Trace de l'algorithme : racine ≠ source

itér [2]

mise à jour de l'ensemble des candidats



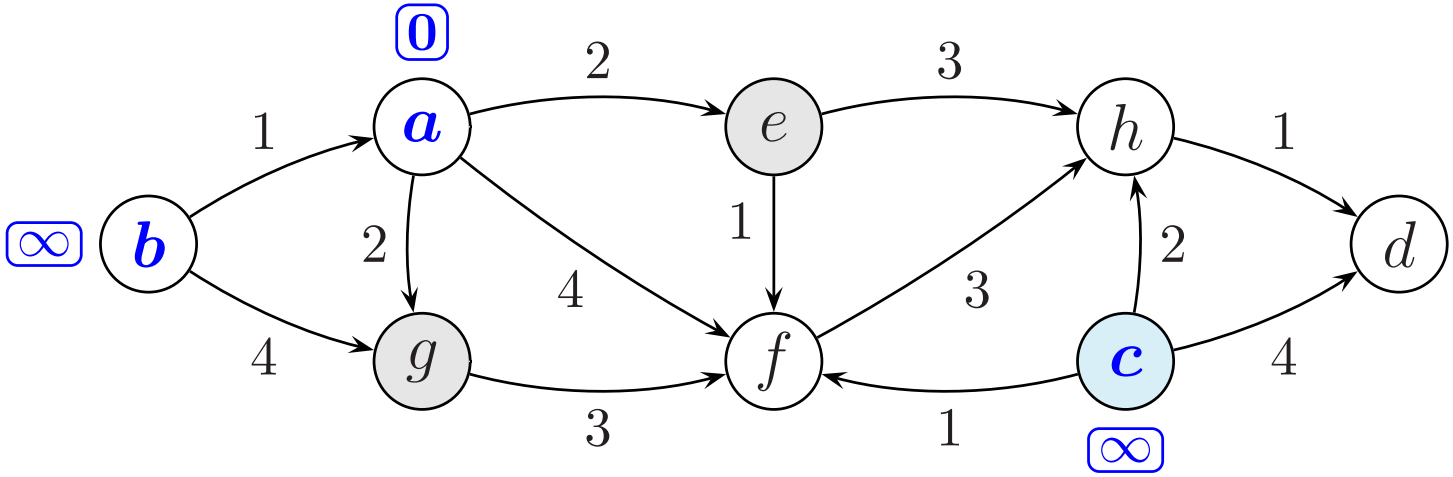
les sommets e et g deviennent candidats

candidats	b, c	a, c	c, e, g
s	b	a	
$\Pi[s]$	∞	0	
$\mathcal{A}[s]$	b	none	

Trace de l'algorithme : racine ≠ source

itér [3]

examiner(c) : aucun arc à relâcher car $V^-(c) = \varnothing$



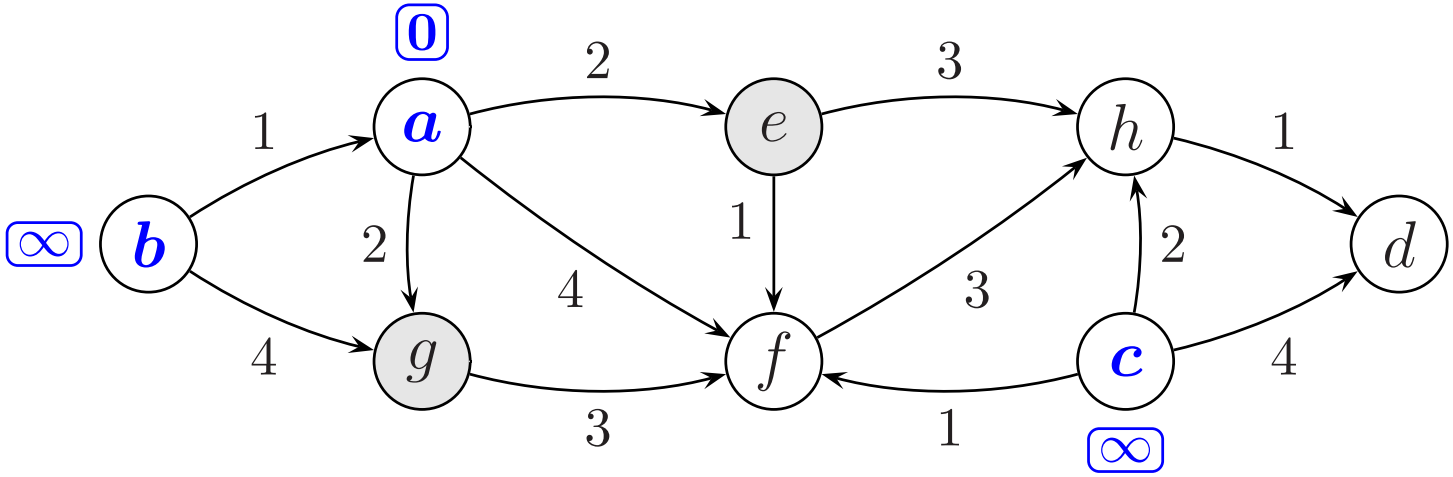
$\Pi[c] \leftarrow \infty \qquad \mathcal{A}[c] \leftarrow c$

candidats	b, c	a, c	c, e, g
s	b	a	c
$\Pi[s]$	∞	0	∞
$\mathcal{A}[s]$	b	none	c

Trace de l'algorithme : racine ≠ source

itér [3]

mise à jour de l'ensemble des candidats



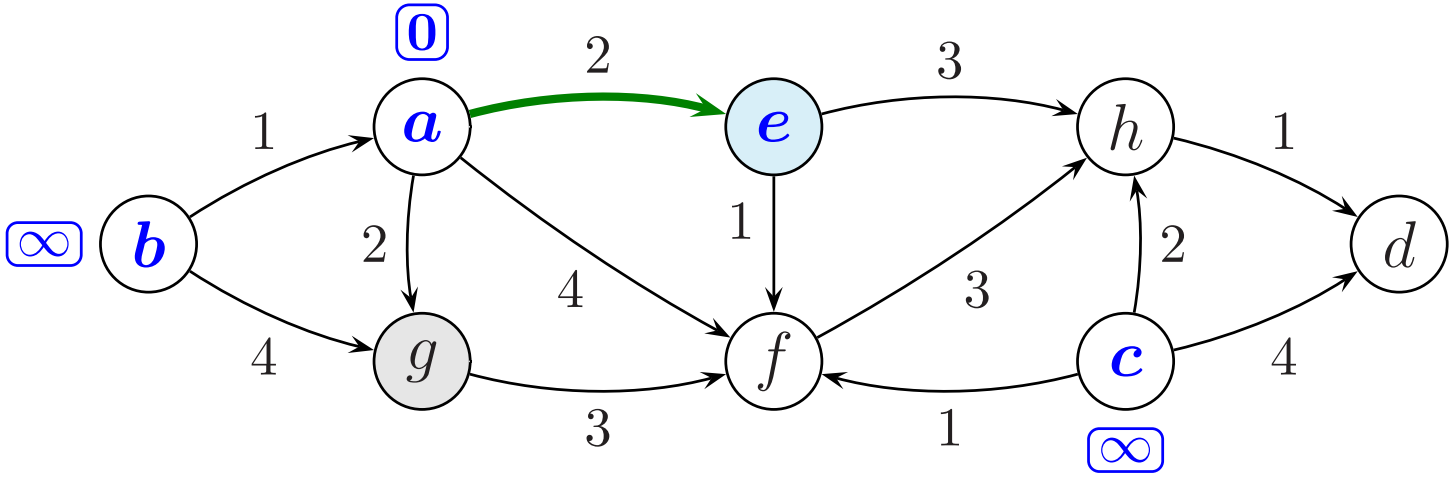
pas de nouveau candidat

candidats	b, c	a, c	c, e, g	e, g
s	b	a	c	
$\Pi[s]$	∞	0	∞	
$\mathcal{A}[s]$	b	none	c	

Trace de l'algorithme : racine ≠ source

itér [4]

examiner(e) : relâcher l'arc (a, e)



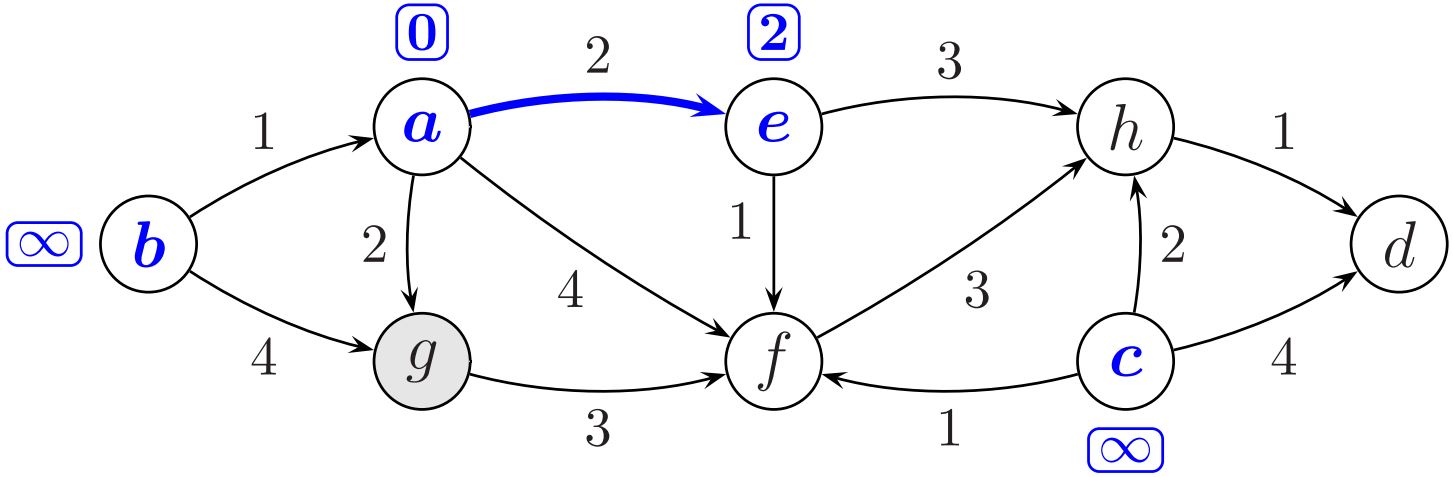
$$\Pi[e] \leftarrow \underbrace{\Pi[a]}_{0} + \underbrace{P(a,e)}_{2} = 2$$

candidats	b, c	a, c	c, e, g	e, g
s	b	a	c	e
$\Pi[s]$	∞	0	∞	2
$\mathcal{A}[s]$	b	none	c	a

Trace de l'algorithme : racine ≠ source

itér [4]

mise à jour de l'ensemble des candidats



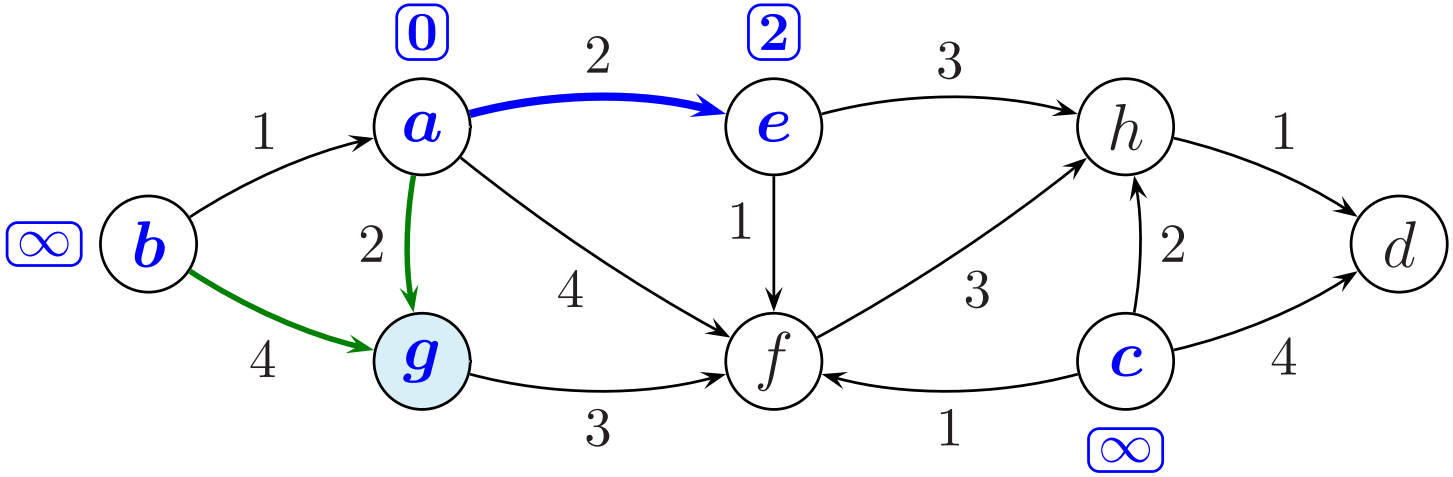
pas de nouveau candidat

candidats	b, c	a, c	c, e, g	e, g	g
s	b	a	c	e	
$\Pi[s]$	∞	0	∞	2	
$\mathcal{A}[s]$	b	none	c	a	

Trace de l'algorithme : racine ≠ source

itér [5]

examiner(g) : relâcher les arcs (a, g) et (b, g)



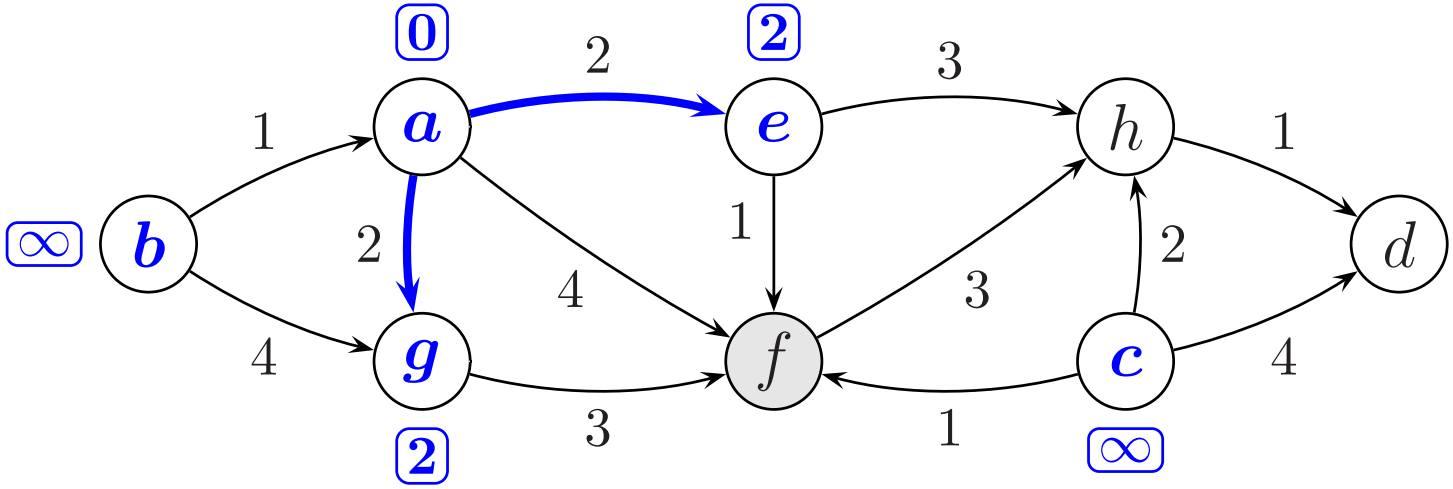
$$\Pi[g] \leftarrow \underbrace{\Pi[a]}_{0} + \underbrace{P(a,g)}_{2} = 2 \qquad \underbrace{\Pi[b]}_{\infty} + \underbrace{P(b,g)}_{4} = \infty$$

candidats	b, c	a, c	c, e, g	e, g	g
s	b	a	c	e	g
$\Pi[s]$	∞	0	∞	2	2
$\mathcal{A}[s]$	b	none	c	a	a

Trace de l'algorithme : racine ≠ source

itér [5]

mise à jour de l'ensemble des candidats



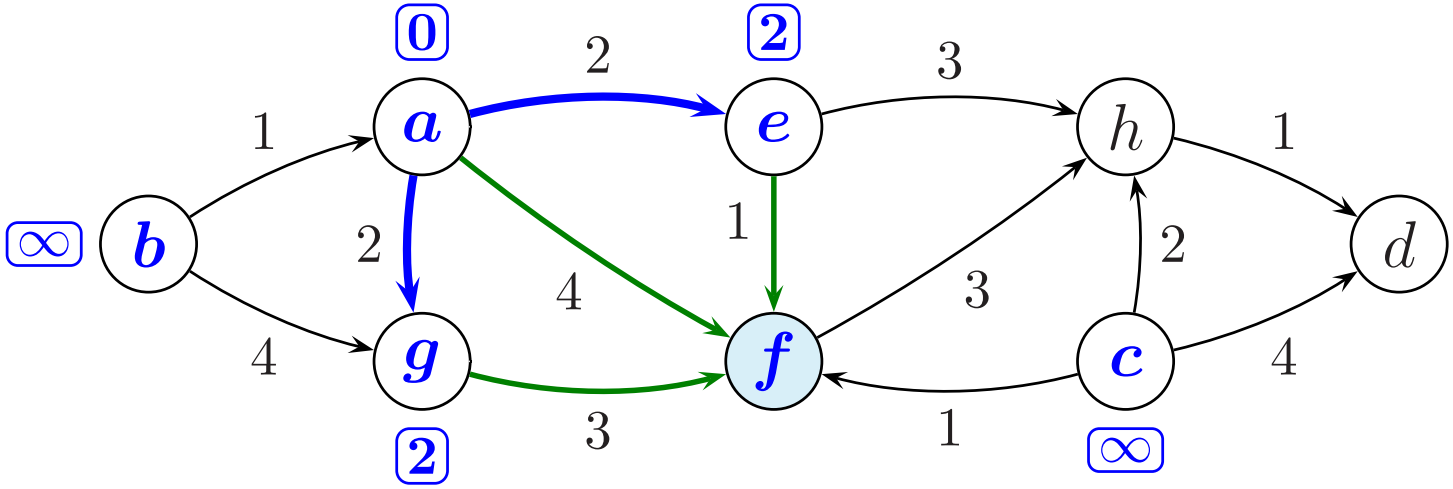
le sommet f devient candidat

candidats	b, c	a, c	c, e, g	e, g	g	f
s	b	a	c	e	g	
$\Pi[s]$	∞	0	∞	2	2	
$\mathcal{A}[s]$	b	none	c	a	a	

Trace de l'algorithme : racine ≠ source

itér [6]

examiner(f) : relâcher les arcs (a, f), (e, f) et (g, f)



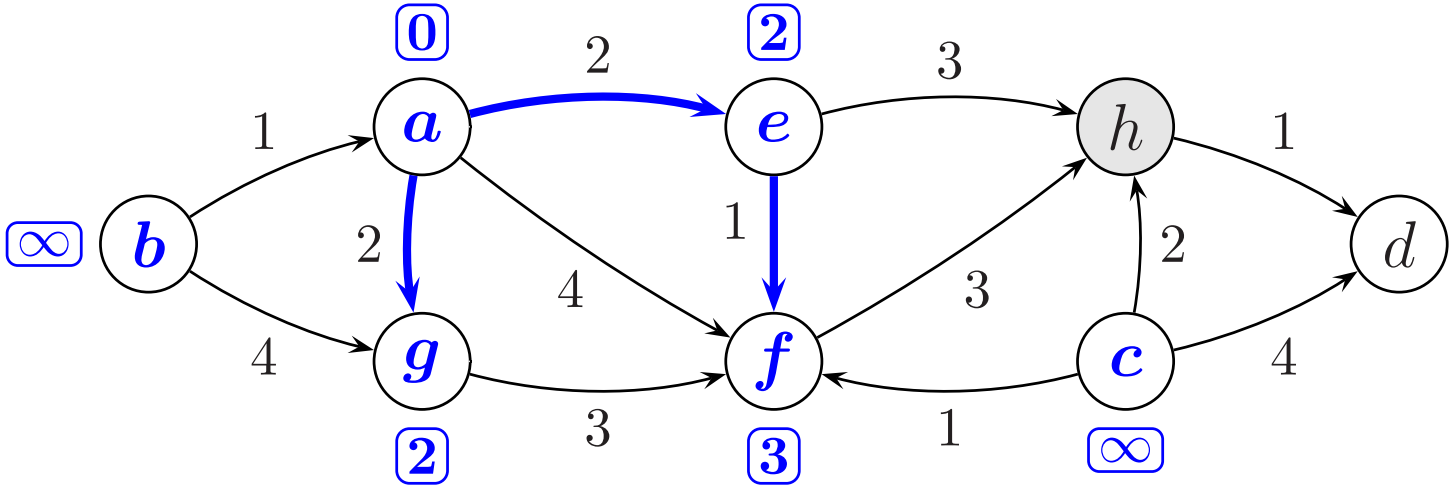
$$\underbrace{\Pi[a]}_{0} + \underbrace{P(a,f)}_{4} = 4 \qquad \Pi[f] \leftarrow \underbrace{\Pi[e]}_{2} + \underbrace{P(e,f)}_{1} = 3 \qquad \underbrace{\Pi[g]}_{2} + \underbrace{P(g,f)}_{3} = 5$$

candidats	b, c	a, c	c, e, g	e, g	g	f
s	b	a	c	e	g	f
$\Pi[s]$	b	0	c	2	2	3
$\mathcal{A}[s]$	b	none	c	a	a	e

Trace de l'algorithme : racine ≠ source

itér [6]

mise à jour de l'ensemble des candidats



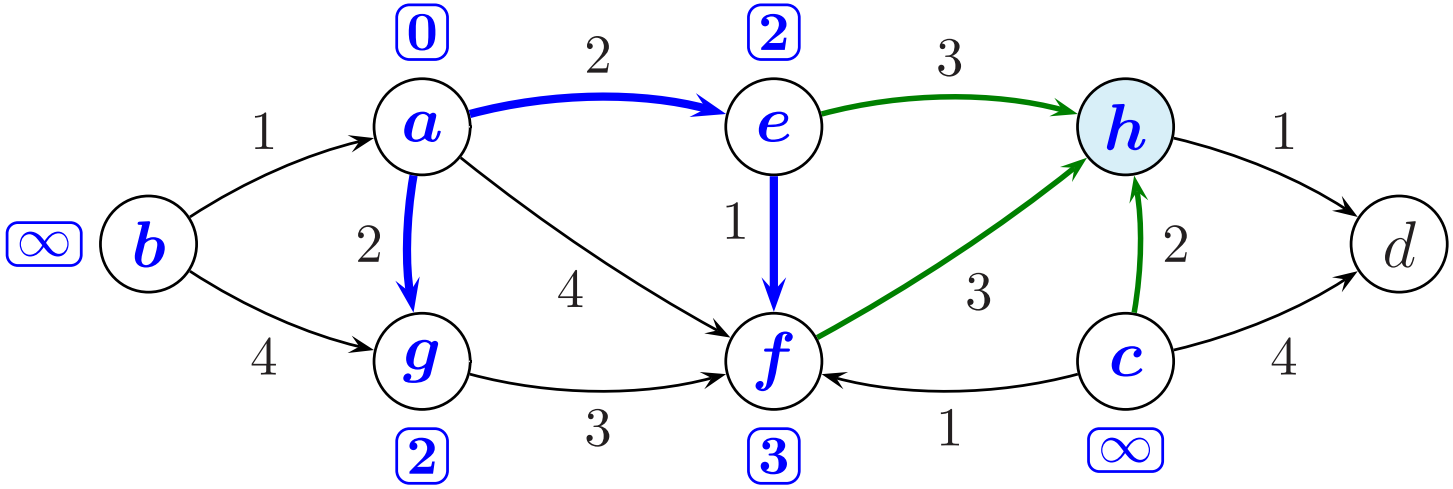
le sommet h devient candidat

candidats	b, c	a, c	c, e, g	e, g	g	f	h
s	b	a	c	e	g	f	
$\Pi[s]$	∞	0	∞	2	2	3	
$\mathcal{A}[s]$	b	none	c	a	a	e	

Trace de l'algorithme : racine ≠ source

itér [7]

examiner(h) : relâcher les arcs (c,h), (e,h) et (f,h)



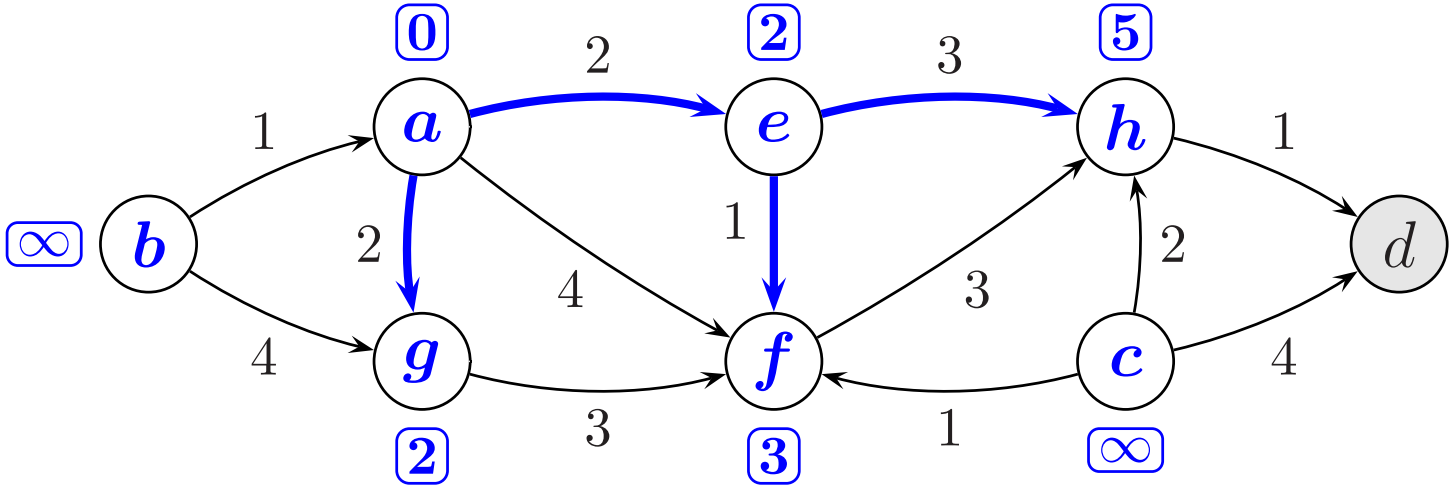
$\underbrace{\Pi[c]}_{\infty} + \underbrace{P(c,h)}_{2} = \infty$ $\quad$ $\Pi[h] \leftarrow \underbrace{\Pi[e]}_{2} + \underbrace{P(e,h)}_{3} = 5$ $\quad$ $\underbrace{\Pi[f]}_{3} + \underbrace{P(f,h)}_{3} = 6$

candidats	b,c	a,c	c,e,g	e,g	g	f	h
s	b	a	c	e	g	f	h
$\Pi[s]$	∞	0	∞	2	2	3	5
$\mathcal{A}[s]$	b	none	c	a	a	e	e

Trace de l'algorithme : racine ≠ source

itér [7]

mise à jour de l'ensemble des candidats



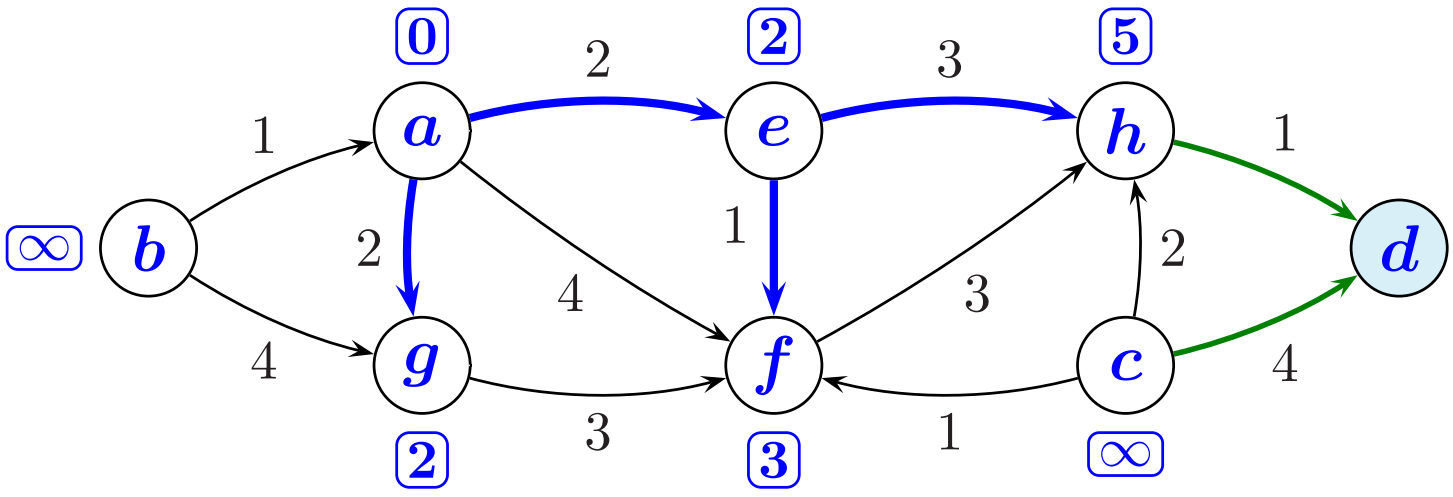
le sommet d devient candidat

candidats	b, c	a, c	c, e, g	e, g	g	f	h	d
s	b	a	c	e	g	f	h	
$\Pi[s]$	∞	0	∞	2	2	3	5	
$\mathcal{A}[s]$	b	none	c	a	a	e	e	

Trace de l'algorithme : racine ≠ source

itér [8]

examiner(d) : relâcher (c, d) et (h, d)



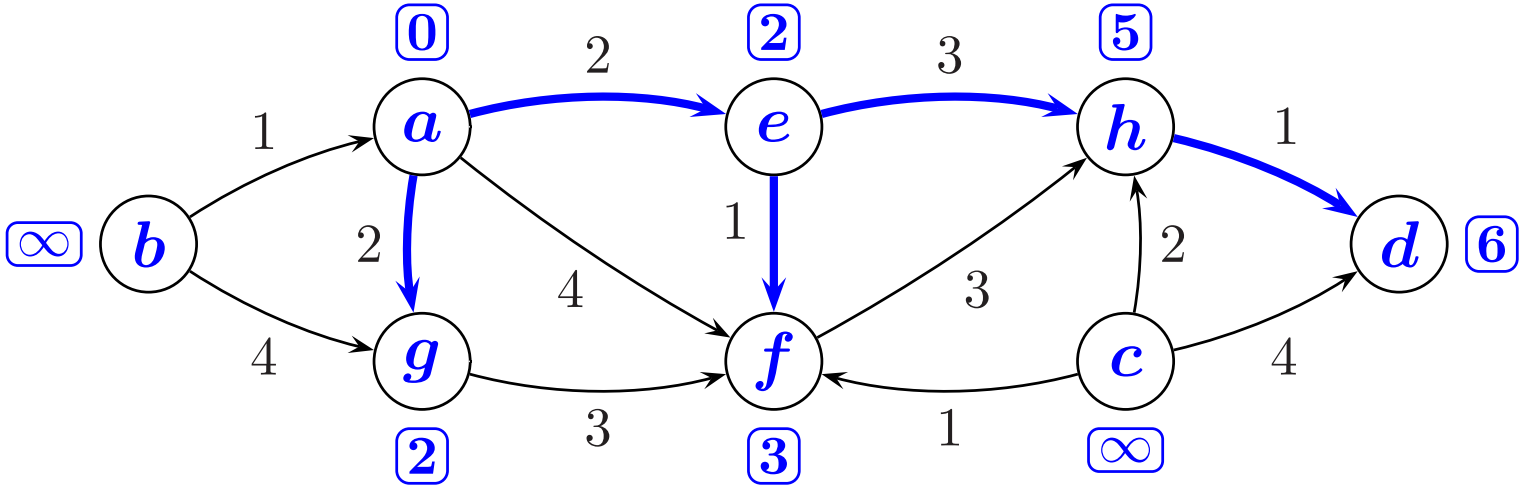
$$\underbrace{\Pi[c]}_{\infty} + \underbrace{P(c,d)}_{4} = \infty \qquad \Pi[d] \leftarrow \underbrace{\Pi[h]}_{5} + \underbrace{P(h,d)}_{1} = 6$$

candidats	b, c	a, c	c, e, g	e, g	g	f	h	d
s	b	a	c	e	g	f	h	d
$\Pi[s]$	∞	0	∞	2	2	3	5	6
$\mathcal{A}[s]$	b	none	c	a	a	e	e	h

Trace de l'algorithme : racine ≠ source

itér [8]

mise à jour de l'ensemble des candidats



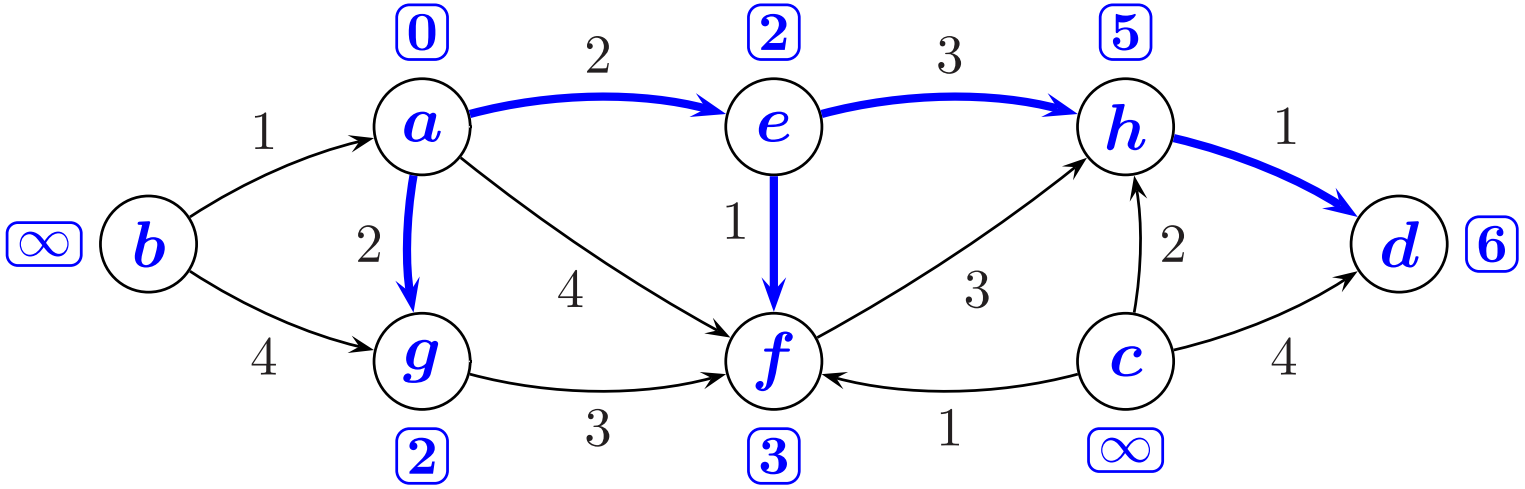
pas de nouveau candidat

candidats	b, c	a, c	c, e, g	e, g	g	f	h	d
s	b	a	c	e	g	f	h	d
$\Pi[s]$	∞	0	∞	2	2	3	5	6
$\mathcal{A}[s]$	b	none	c	a	a	e	e	h

Trace de l'algorithme : racine ≠ source

fin

l'ensemble des candidats est vide : fin de l'algorithme



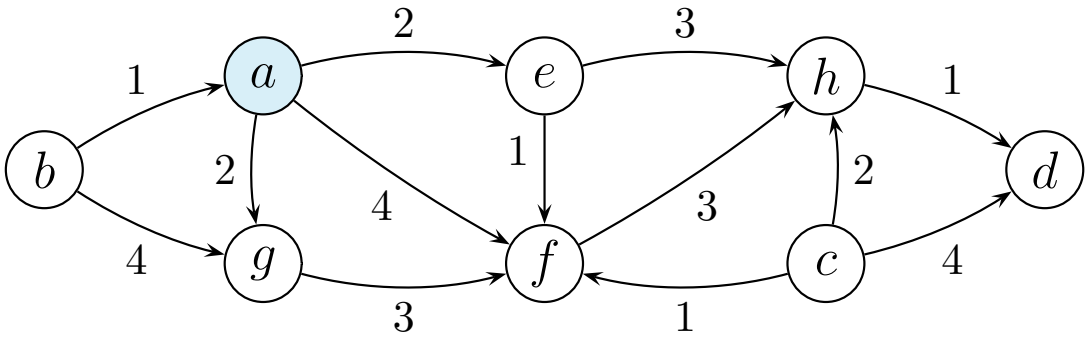
a-arborescence de cpm

candidats	b, c	a, c	c, e, g	e, g	g	f	h	d
s	b	a	c	e	g	f	h	d
$\Pi[s]$	∞	0	∞	2	2	3	5	6
$\mathcal{A}[s]$	b	none	c	a	a	e	e	h

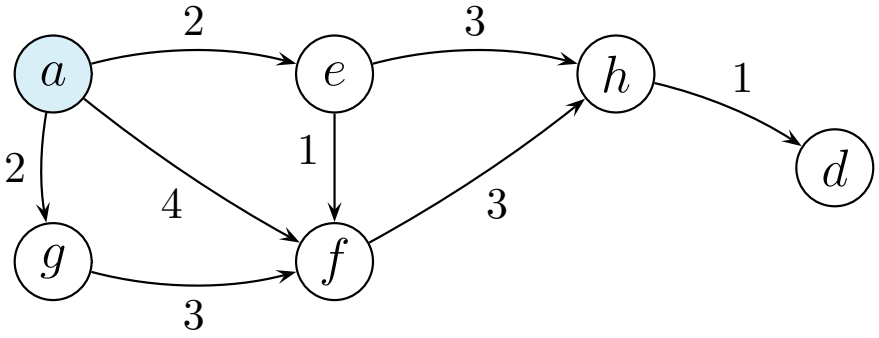
Autre possibilité

Si r n'est pas l'unique source de G, on commence par construire le sous-graphe de G induit par l'ensemble des descendants de r. On applique ensuite l'algorithme de Bellman à ce sous-graphe.

graphe initial
$r = a$



sous-graphe de
source unique
$r = a$



5. VARIANTES

- Rechercher une r-arborescence de chemins de poids maximum.
  (calcul des dates de début au plus tôt dans les méthodes PERT et MPM)

- Rechercher une r-arborescence de chemins de probabilité maximum. Quand les poids des arcs sont des probabilités, le poids d'un chemin doit être calculé comme le produit des poids des arcs qui composent le chemin.
  (fiabilité dans des réseaux)

- Rechercher tous les chemins de poids minimum (ou maximum) aboutissant en un sommet r. Le résultat obtenu est alors une anti-arborescence de cpm d'anti-racine r. Pour ce faire, Il suffit d'examiner les sommets dans l'ordre inverse d'un ordre topologique.
  (utilisé pour le calcul des dates de début au plus tard dans les méthodes PERT et MPM)